Firebase CRUD Operations in Dart

Cloud Database Programming Tutorial

Learning Objectives

By the end of this session, you will understand:

- ✅ **Firebase Firestore** NoSQL database concepts
- ✅ **CRUD** operations in cloud databases
- ✅ **Real-time data** synchronization
- ✅ **Document-based** data modeling
- ✅ **Firebase vs SQLite** trade-offs
- ✅ **Testing** with fake cloud services

What is Firebase Firestore?

Firebase Firestore is a NoSQL document database in the cloud:

Feature	Description	Benefit
Documents	JSON-like data structure	Flexible schema
Collections	Groups of documents	Organized data
Real-time	Live data synchronization	Instant updates
Offline	Works without internet	Better UX
Scalable	Auto-scaling infrastructure	Handles growth

Key Difference: Documents (JSON) vs Tables (SQL)

Project Structure Overview

```
firebase/
├── lib/
│   ├── models/
│   │   └── foobar.dart                    # Data model with JSON support
│   └── services/
│       └── foobar_crud_firebase.dart      # Firebase CRUD operations
├── test/
│   └── foobar_crud_test.dart              # Tests with fake Firebase
├── doc/
│   └── firebase_tutorial.md               # This presentation
└── run.sh                                 # Easy run script
```

Key Insight: Same structure as SQLite but different implementations!

The FooBar Model - Firebase Version

```dart
class FooBar {
  String? id;          // Firebase document ID (String, not int!)
  String foo;
  int bar;
  DateTime? createdAt; // Server timestamp support

  FooBar({this.id, required this.foo, required this.bar, this.createdAt});

  // Convert from Firebase document
  factory FooBar.fromDocument(DocumentSnapshot doc) {
    final data = doc.data() as Map<String, dynamic>;
    return FooBar(
      id: doc.id,  // Auto-generated Firebase ID
      foo: data['foo'] as String,
      bar: data['bar'] as int,
      createdAt: (data['createdAt'] as Timestamp?)?.toDate(),
    );
  }

  // Convert to JSON for Firebase storage
  Map<String, dynamic> toJson() => {
    'foo': foo,
    'bar': bar,
    // createdAt added automatically by server
  };
}
```

Firebase vs SQLite: Data Structure

SQLite (Relational)

```
CREATE TABLE foobars (
  id INTEGER PRIMARY KEY AUTOINCREMENT,
  foo TEXT NOT NULL,
  bar INTEGER NOT NULL,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

Firebase (Document)

```
{
  "foobars": {
    "auto-generated-id-1": {
      "foo": "Hello",
      "bar": 42,
      "createdAt": "2024-01-15T10:30:00Z"
    },
```

Firebase Setup and Initialization

```dart
class FooBarCrudFirebase {
  final FirebaseFirestore _firestore;
  final String _collectionName = 'foobars';

  // Constructor allows dependency injection (great for testing!)
  FooBarCrudFirebase({FirebaseFirestore? firestore})
      : _firestore = firestore ?? FirebaseFirestore.instance;

  // Get collection reference
  CollectionReference get _collection =>
      _firestore.collection(_collectionName);
}
```

Educational Note: Constructor injection enables testing with fake Firebase!

CREATE Operation - Firebase Style

```
Future<String> create(FooBar foobar) async {
  try {
    // Prepare data with server timestamp
    final data = foobar.toJson();
    data['createdAt'] = FieldValue.serverTimestamp(); // Firebase magic!

    // Add to collection (auto-generates ID)
    final docRef = await _collection.add(data);

    return docRef.id; // Returns Firebase document ID (String)
  } catch (e) {
    throw Exception('Failed to create FooBar: $e');
  }
}
```

Key Differences from SQLite:

- Returns `String` ID instead of `int`

READ Operations - Multiple Approaches

Read All Documents

```
Future<List<FooBar>> readAll() async {
  final querySnapshot = await _collection
      .orderBy('createdAt', descending: false)  // Sort by timestamp
      .get();

  return querySnapshot.docs
      .map((doc) => FooBar.fromDocument(doc))
      .toList();
}
```

Read by ID

```
Future<FooBar?> read(String id) async {
  final docSnapshot = await _collection.doc(id).get();

  if (!docSnapshot.exists) return null;
```

Firebase Queries vs SQL

Exact Match (Easy)

```
Future<List<FooBar>> findByFoo(String foo) async {
  final querySnapshot = await _collection
      .where('foo', isEqualTo: foo)  // Simple equality
      .get();

  return querySnapshot.docs.map((doc) => FooBar.fromDocument(doc)).toList();
}
```

Range Queries (Firebase Strength)

```
Future<List<FooBar>> findByBarRange(int minBar, int maxBar) async {
  final querySnapshot = await _collection
      .where('bar', isGreaterThanOrEqualTo: minBar)
      .where('bar', isLessThanOrEqualTo: maxBar)
      .orderBy('bar')
      .get();
```

Firebase Query Limitations

Text Search Challenge

```
// ❌ Firebase doesn't have SQL LIKE operator
// Can't do: WHERE foo LIKE '%search%'

// ✅ Workaround: Client-side filtering
Future<List<FooBar>> findByFooContains(String searchText) async {
  final allFooBars = await readAll();  // Get all docs

  return allFooBars
      .where((foobar) =>
          foobar.foo.toLowerCase().contains(searchText.toLowerCase()))
      .toList();
}
```

Trade-off: Simplicity vs Query Power

- Firebase: Simple queries, real-time updates

UPDATE Operations

Full Document Update

```
Future<bool> update(String id, FooBar foobar) async {
  final docRef = _collection.doc(id);

  // Check existence first
  final docSnapshot = await docRef.get();
  if (!docSnapshot.exists) return false;

  // Update with timestamp
  final data = foobar.toJson();
  data['updatedAt'] = FieldValue.serverTimestamp();

  await docRef.update(data);
  return true;
}
```

Partial Field Update (Efficient)

DELETE Operations

Single Document

```
Future<bool> delete(String id) async {
  final docRef = _collection.doc(id);

  final docSnapshot = await docRef.get();
  if (!docSnapshot.exists) return false;

  await docRef.delete();
  return true;
}
```

Delete All (Batch Operation)

```
Future<int> deleteAll() async {
  final querySnapshot = await _collection.get();
  final batch = _firestore.batch();
```

Real-time Features (Firebase's Superpower!)

Stream All Documents

```
Stream<List<FooBar>> streamAll() {
  return _collection
      .orderBy('createdAt')
      .snapshots()  // Real-time listener!
      .map((querySnapshot) => querySnapshot.docs
          .map((doc) => FooBar.fromDocument(doc))
          .toList());
}
```

Stream Single Document

```
Stream<FooBar?> streamById(String id) {
  return _collection
      .doc(id)
      .snapshots()
      .map((docSnapshot) {
```

Pagination - Handling Large Datasets

```
Future<List<FooBar>> readPaginated({
  int limit = 10,
  DocumentSnapshot? startAfter,
}) async {
  Query query = _collection
      .orderBy('createdAt')
      .limit(limit);

  if (startAfter != null) {
    query = query.startAfterDocument(startAfter);  // Continue from here
  }

  final querySnapshot = await query.get();
  return querySnapshot.docs
      .map((doc) => FooBar.fromDocument(doc))
      .toList();
}
```


Educational Value: Shows how to handle large datasets efficiently

Batch Operations (Performance Optimization)

```
Future<List<String>> createBatch(List<FooBar> foobars) async {
  final batch = _firestore.batch();
  final docIds = <String>[];

  for (final foobar in foobars) {
    final docRef = _collection.doc(); // Pre-generate ID
    final data = foobar.toJson();
    data['createdAt'] = FieldValue.serverTimestamp();

    batch.set(docRef, data);
    docIds.add(docRef.id);
  }

  await batch.commit(); // All operations happen atomically
  return docIds;
}
```

Benefit: Faster than individual operations, atomic transactions

Testing with Fake Firebase

```dart
import 'package:fake_cloud_firestore/fake_cloud_firestore.dart';

void main() {
  late FooBarCrudFirebase crudService;
  late FakeFirebaseFirestore fakeFirestore;

  setUp(() {
    fakeFirestore = FakeFirebaseFirestore(); // Mock Firebase
    crudService = FooBarCrudFirebase(firestore: fakeFirestore);
  });

  test('CREATE: Should add a new document', () async {
    final id = await crudService.create(testFooBar);
    expect(id, isNotEmpty);

    final retrieved = await crudService.read(id);
    expect(retrieved!.foo, equals(testFooBar.foo));
  });
}
```

Firebase vs SQLite Comparison

Aspect	SQLite	Firebase
Storage	Local file	Cloud database
Data Model	Relational (tables/rows)	Document (collections/docs)
Queries	Full SQL power	Limited but fast
Real-time	Manual refresh	Automatic streams
Offline	Always offline	Smart offline sync
Scaling	Single device	Auto-scaling cloud
Cost	Free (local)	Pay-per-usage
Setup	Simple	Requires configuration

When to Choose Firebase vs SQLite

Choose Firebase When:

- ✅ Need real-time synchronization
- ✅ Building mobile/web apps
- ✅ Want automatic scaling
- ✅ Need multi-user collaboration
- ✅ Want offline-online sync
- ✅ Rapid prototyping

Choose SQLite When:

- ✅ Single-user applications
- ✅ Complex relational queries
- ✅ Offline-first applications

Security Considerations

Firebase Security Rules (Server-side)

```
rules_version = '2';
service cloud.firestore {
  match /databases/{database}/documents {
    match /foobars/{document} {
      allow read, write: if request.auth != null; // Authenticated users only
    }
  }
}
```

SQLite Security (Client-side)

```
// File system permissions
// Application-level access control
// Database encryption (if needed)
```

Trade-off: Firebase has built-in cloud security, SQLite relies on app security

Performance Considerations

Firebase Optimization Tips

```
// ✅ GOOD: Use pagination for large datasets
final results = await readPaginated(limit: 20);

// ✅ GOOD: Use specific field updates
await updateFields(id, {'bar': newValue});

// ✅ GOOD: Use batch operations
await createBatch(multipleFooBars);

// ❌ AVOID: Reading all documents for simple searches
final all = await readAll(); // Then filter client-side
```

SQLite Optimization Tips

```
// ✅ GOOD: Use prepared statements
final stmt = db.prepare('SELECT * FROM foobars WHERE id = ?');
```

Error Handling Patterns

Firebase Error Handling

```
Future<FooBar?> safeRead(String id) async {
  try {
    return await read(id);
  } on FirebaseException catch (e) {
    print('Firebase error: ${e.code} - ${e.message}');
    return null;
  } catch (e) {
    print('Unexpected error: $e');
    return null;
  }
}
```

Common Firebase Exceptions

- `permission-denied` : Security rules violation
- `unavailable` : Network connectivity issues

Running the Firebase Project

1. Quick Start

```
cd firebase/
./run.sh                # Run everything
./run.sh test            # Run tests only
./run.sh demo            # Run demo only
```

2. Manual Commands

```
dart pub get             # Install dependencies
dart test                # Run comprehensive tests
dart run lib/main.dart # Run the demo
```

3. Test Output Analysis

```
dart test --reporter=expanded  # Detailed test output
```

Real Firebase Setup (Advanced)

1. Create Firebase Project

- Visit Firebase Console
- Create new project
- Enable Firestore Database

2. Add Firebase to Your App

```
dart pub global activate flutterfire_cli
flutterfire configure
```

3. Initialize in Code

```
import 'package:firebase_core/firebase_core.dart';
import 'firebase_options.dart';
```

Advanced Firebase Features

1. Compound Queries

```
// Multiple where clauses
await _collection
    .where('bar', isGreaterThan: 50)
    .where('foo', isEqualTo: 'Active')
    .get();
```

2. Array and Map Queries

```
// Query array fields
await _collection
    .where('tags', arrayContains: 'important')
    .get();
```

3. Subcollections

Best Practices Summary

📝 Data Modeling

- Keep documents flat when possible
- Use subcollections for 1-to-many relationships
- Denormalize data for read performance

🔍 Queries

- Create indexes for complex queries
- Use pagination for large result sets
- Prefer specific field updates over full document updates

🔒 Security

- Always write security rules

Common Pitfalls and Solutions

1. Query Limitations

```
// ❌ Can't do complex text search
// ✅ Use client-side filtering or external search service

// ❌ Can't query across collections easily
// ✅ Denormalize data or use subcollections
```

2. Real-time Listener Management

```
// ❌ Forgetting to cancel listeners
StreamSubscription? subscription;

// ✅ Always cancel when done
subscription = streamAll().listen(onData);
// Later: subscription?.cancel();
```

Assignment Ideas

Beginner Level

1. **Message Board**: Real-time messages with timestamps
2. **Todo List**: Tasks with real-time synchronization
3. **Simple Chat**: Messages between users

Intermediate Level

1. **Expense Tracker**: Categories, amounts, real-time totals
2. **Collaborative Notes**: Multiple users editing documents
3. **Event RSVP System**: Real-time participant tracking

Advanced Level

1. **Social Media Feed**: Posts, comments, likes with real-time updates

Integration with Other Technologies

Flutter Mobile Apps

```
// Firebase works seamlessly with Flutter
StreamBuilder<List<FooBar>>(
  stream: crudService.streamAll(),
  builder: (context, snapshot) {
    if (snapshot.hasData) {
      return ListView(
        children: snapshot.data!.map((foobar) =>
          ListTile(title: Text(foobar.foo))
        ).toList(),
      );
    }
    return CircularProgressIndicator();
  },
)
```

Web Applications

Performance Monitoring

Firebase Performance

- Built-in performance monitoring
- Real-time usage statistics
- Automatic scaling based on load

Monitoring Code

```
// Add performance tracking
final stopwatch = Stopwatch()..start();
await crudService.readAll();
stopwatch.stop();
print('Operation took: ${stopwatch.elapsedMilliseconds}ms');
```

Educational Value: Understanding performance implications of database choices

Future Learning Paths

1. Firebase Ecosystem

- Firebase Authentication
- Firebase Functions (serverless)
- Firebase Storage (file uploads)
- Firebase Hosting

2. Advanced Database Concepts

- Database indexing strategies
- Real-time system architecture
- Event-driven programming
- CQRS (Command Query Responsibility Segregation)

Summary & Key Takeaways

✅ What You've Learned

- Firebase provides powerful real-time capabilities
- Document databases offer flexibility at the cost of query complexity
- Testing with mocks enables rapid development
- Cloud databases have different trade-offs than local databases

🚀 Key Firebase Advantages

- **Real-time synchronization** out of the box
- **Automatic scaling** without server management
- **Offline support** with intelligent sync
- **Cross-platform** compatibility

Resources & References

Documentation

- Firebase Documentation
- Firestore Documentation
- Flutter Firebase Documentation

Testing Tools

- fake_cloud_firestore
- Firebase Emulator Suite

Best Practices

- Firestore Data Modeling
- Firestore Security Rules

Questions & Discussion

Comparison Questions

1. When would you choose Firebase over SQLite and vice versa?
2. How do real-time features change application architecture?
3. What are the security implications of cloud vs local databases?

Technical Questions

1. How would you implement complex searches in Firebase?
2. How do you handle offline scenarios in real-time apps?
3. What are the cost implications of Firebase queries?

Thank you for your attention!

Ready for hands-on Firebase development? 🔥